



ÓBUDAI EGYETEM
ÓBUDA UNIVERSITY

Korszerű adatbázisok előadás 01



ÓBUDAI EGYETEM
ÓBUDA UNIVERSITY

Bevezetés

Tematika

- A Group By speciális lehetőségei
- Részösszegek, végösszegek
- Analitikus függvények
- Lekérdezés optimalizálás
- Programozás SQL-ben
- Speciális SQL adattípusok
- Dokumentum adatbázisok
- Gráf adatbázisok
- Kulcs-érték adatbázisok
- Oszlop-alapú adatbázisok

+



Kapcsolódó tantárgyak/területek

Szoftverfejlesztés

Üzleti
intelligencia

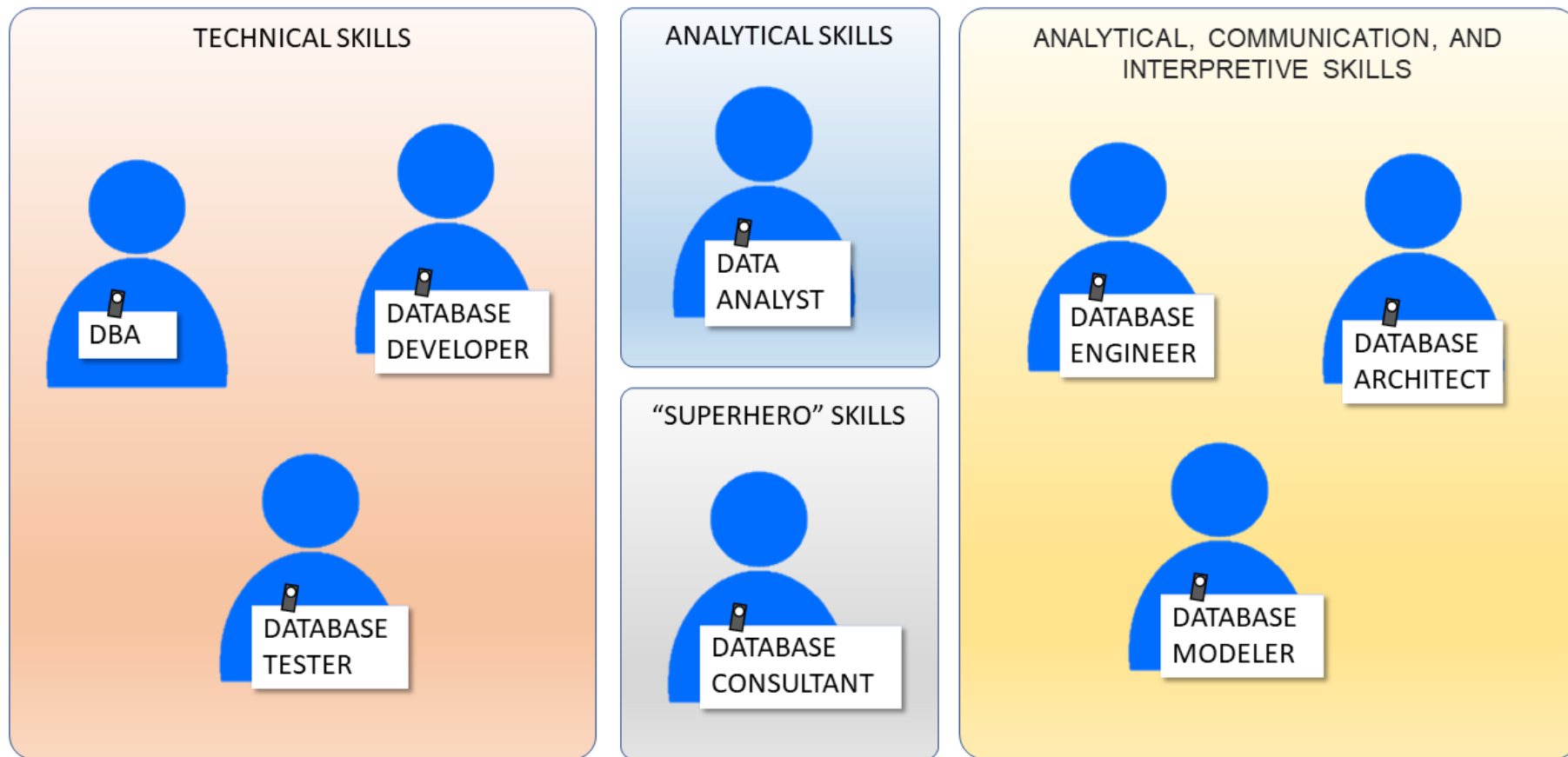
Adatelemzés,
adattudomány

Információs
rendszerek

Döntéstámogatás

IT Architektúrák

Hol alkalmazható a megszerzett tudás?



A kapcsolódó területek pozíciói, pl: webfejlesztő, statisztikus, gazdaságinformatikus stb.

Miért van szükségünk adatbázisokra?



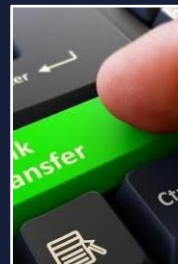
Nagy
adatmennyiség



Sok
felhasználó



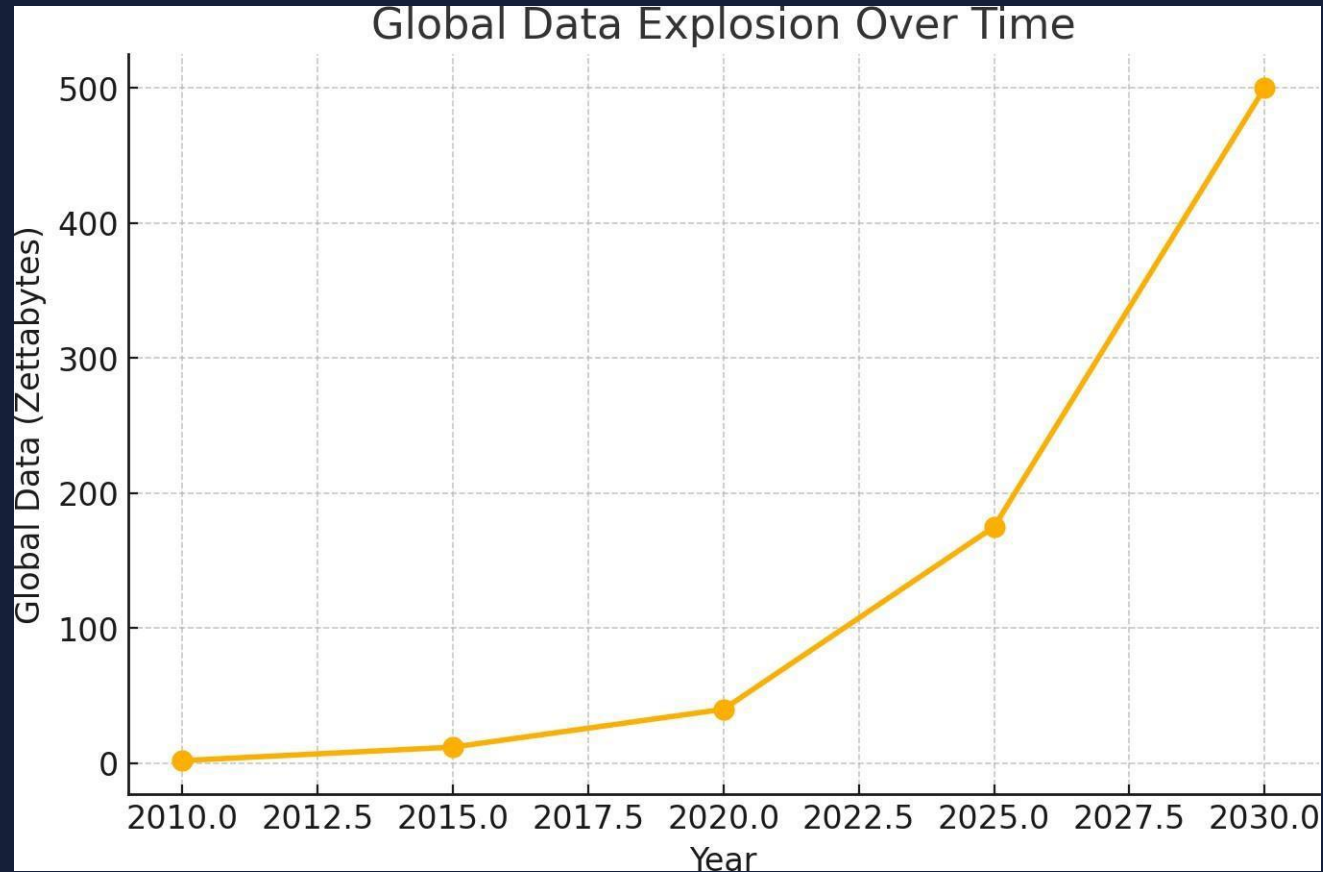
Konziszten-
cia



Adatok
elemzése

Fontos: az adatok lekérdezhetőek legyenek!

Nagy adatmennyiség

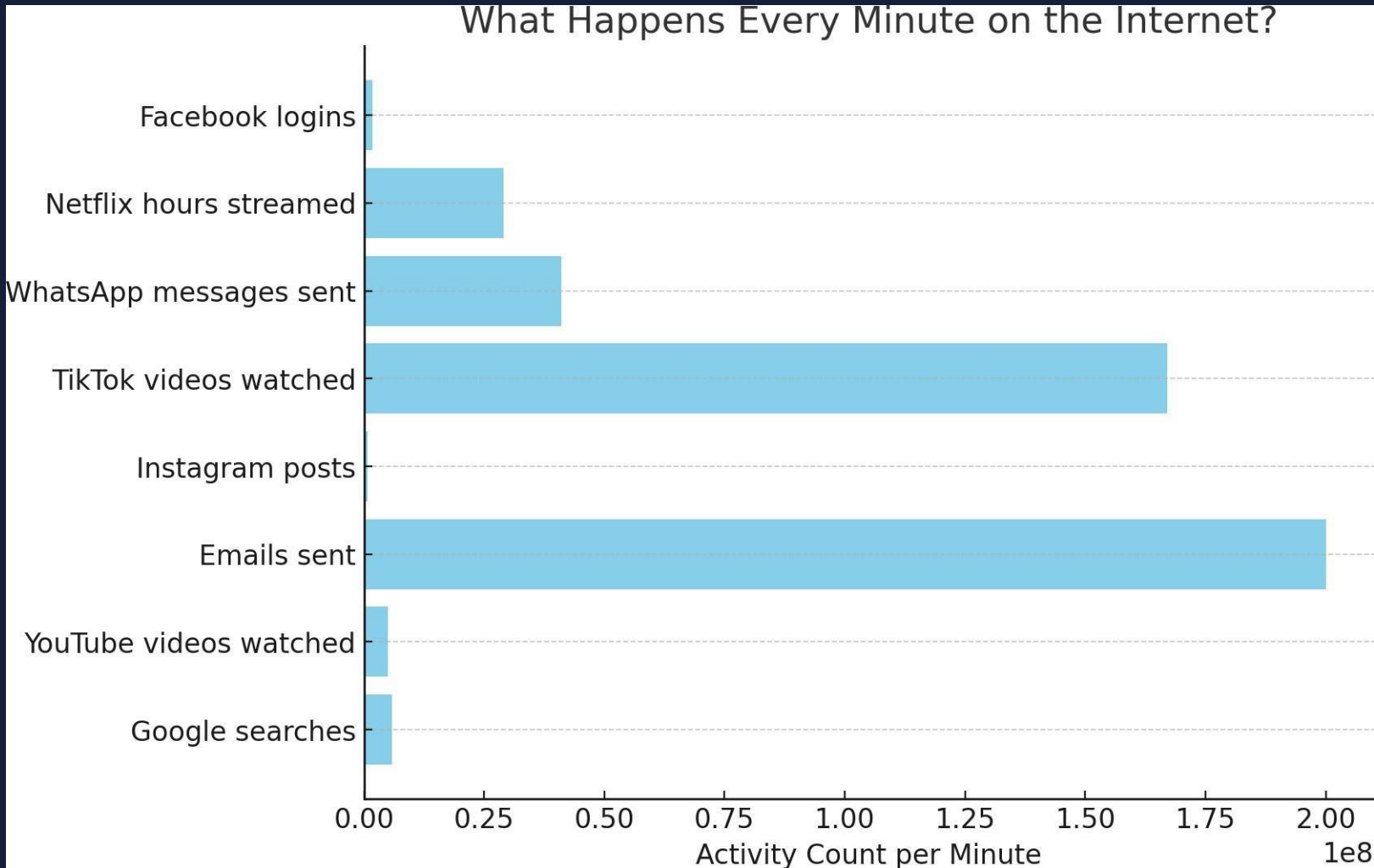


2020: 40 zettabyte (ZB)

2025 előrejelzés: 175 ZB

1 ZB = 1 milliárd terabyte = 1 billió gigabyte

Nagy adatmennyiség (folyt.)



Equivalent Units

DVD (4.7 GB)	1 089 362
USB Flash Drive (128 GB)	40 000
SSD (1 TB)	5 000
HDD (10 TB)	500
Data Center Server (100 TB)	50
Human Brain Memory (2.5 PB)	2 000

Konzisztencia

A **konzisztencia** azt jelenti, hogy az adatok mindig érvényes állapotban maradnak, megfelelnek az előírt szabályoknak és integritási feltételeknek.

Példák:

- ☐ Átutalunk 20 ezer forintot. Konzisztens állapot: az utaló egyenlege 20 ezer forinttal csökken, a fogadóé 20 ezer forinttal nő.
- ☐ Egy webshopban egy adott termékből 5 db van készleten. Konzisztens állapot: ha eladunk egy vagy több terméket, akkor a készlet ennek megfelelően csökken. A készleten lévőnél több termék nem adható el.

Mi történik, ha nincs konzisztencia? (inkonzisztencia)

Következmények:

- ☐ Ellentmondó, hibás adatok jelenhetnek meg
- ☐ Az üzleti folyamatok sérülnek
- ☐ Pénzügyi vagy működési kár keletkezhet

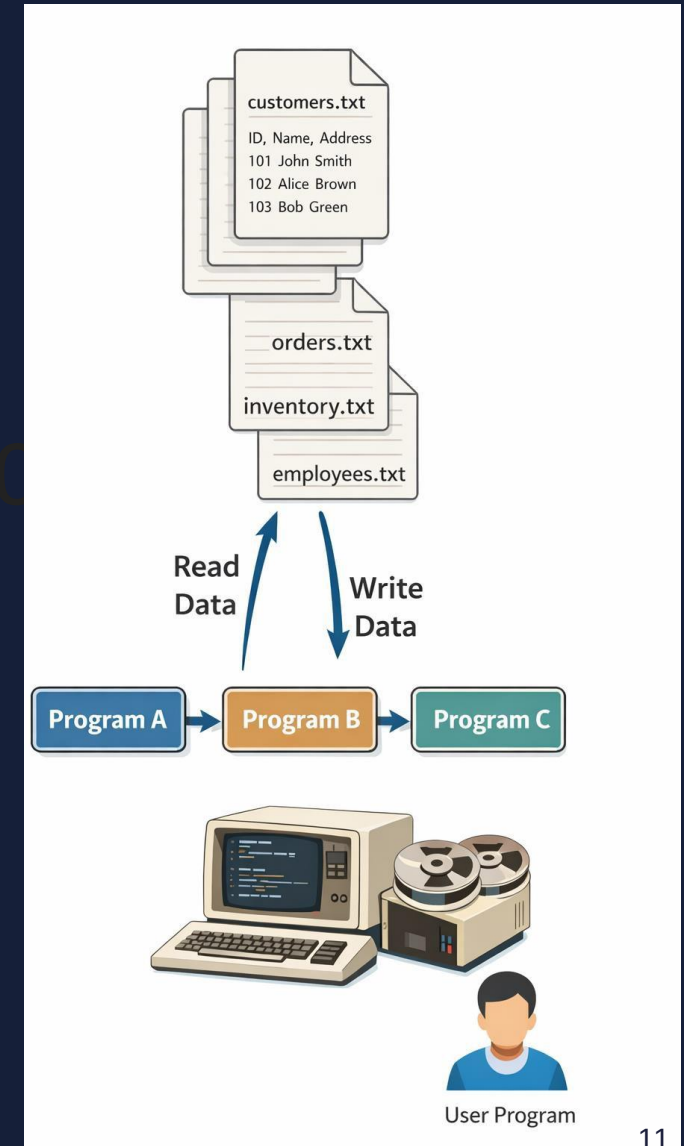
Példák

- ☐ **Banki Tranzakciók.** Ha egy pénzáttétel közben elakad az adatfrissítés, a pénz eltűnhet. Adatbázis esetén ez nem fordulhat elő.
- ☐ **Webshop Rendelések.** Ha a raktárkészlet nincs frissítve, egy vásárló fizethet egy már elfogyott termékért. Az adatbázis biztosítja, hogy a rendelési és készletadatok együtt frissüljenek.
- ☐ A **papíralapú nyilvántartás szétesik**, ha több ember egyszerre dolgozik rajta

Adatbázis nélkül az inkonzisztencia nagyon gyakori — főleg több felhasználó esetén.

Történeti áttekintés - A kezdetek(1960 előtt)

- Még nem voltak adatbázisok
- Az adatokat a felhasználói programok kezelték
 - Az adatokat egyszerű fájlokban (pl. szöveges fájl) tárolták
 - A tárolás sémája esetleges volt
 - Lekérdezni csak a programból lehetett
- Minden egyes adattárolási és lekérdezési feladatra külön programot kellett írni



1960-as évek eleje

Egyre
problémásabb
adatkezelés

Nem mindegy,
hogyan
tároljuk az
adatokat

Szükség van
valamilyen
logikára

Megjelentek
az első
adatmodellek

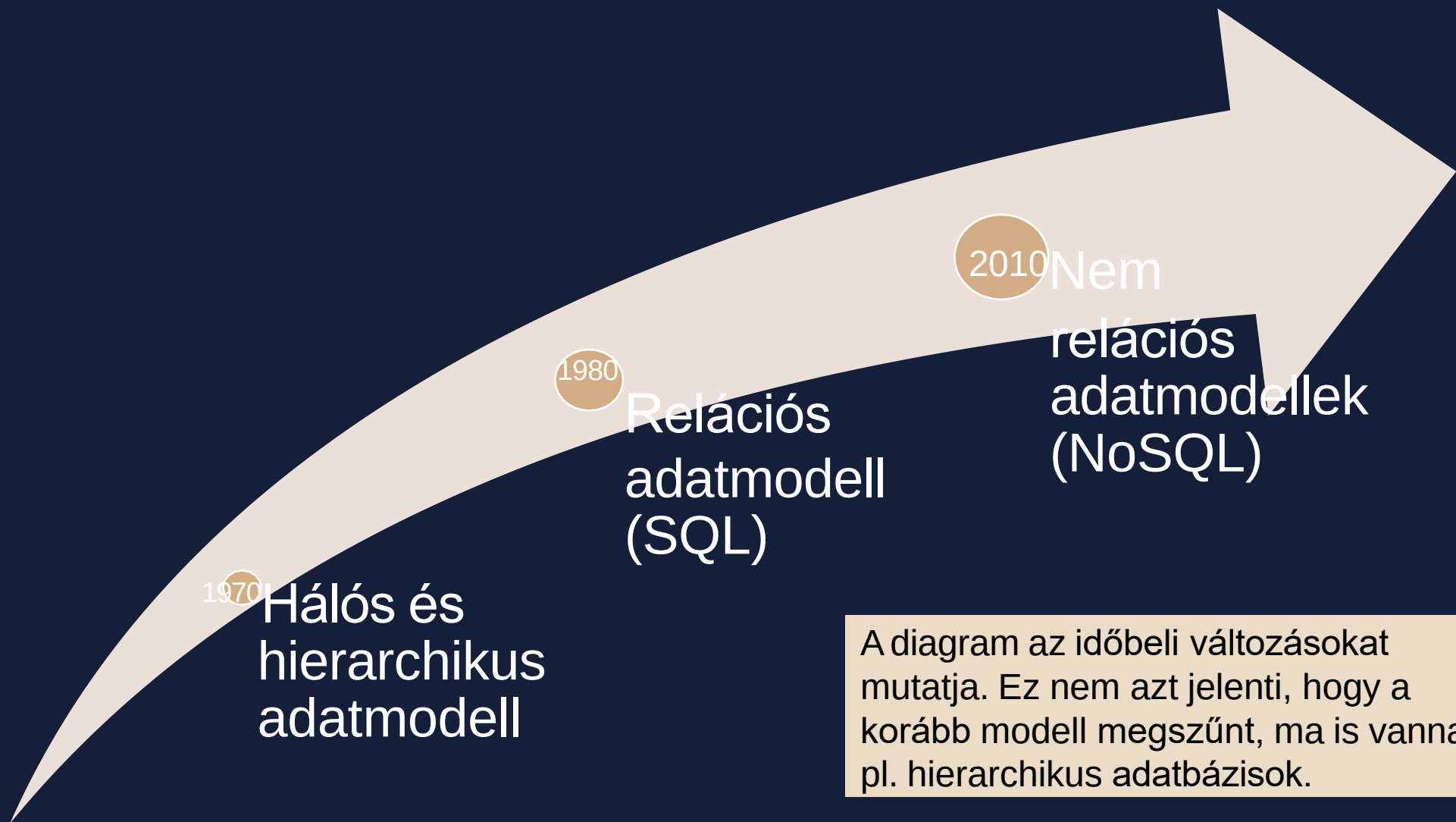
Az 1960-as évek

- Az első adatbázisok és adatmodellek megjelenése
 - IDS (Integrated Data Store)
 - Az egyik legelső adatbáziskezelő
 - Charles Bachman fejlesztette ki
 - CODASYL
 - A hálós adatmodell megfogalmazása
 - COBOL programozók fejlesztése
 - IMS (Information Management System)
 - IBM fejlesztése
 - Hierarchikus adatmodellt használt

Az 1960-as évek után

- Sorra jelentek meg az új adatmodellek, és ezekre épülve az új adatbáziskezelő rendszerek
- Minden adatmodellnek voltak közös elemei
 - Olyan dolgok, amelyekről adatokat tárolunk (egyedek)
 - Az adott dolgok jellemzői (tulajdonságok)
 - Az adott dolgok közötti összefüggések (kapcsolatok)
- **A domináns adatmodell a relációs lett**
- Napjainkban megjelentek és kezdenek elterjedni a nem relációs (NoSQL) adatbázisok

Az adatmodellek fejlődésének mérföldkövei



Mi történt 1980 és 2010 között?

- A relációs adatmodell egyeduralkodó lett
 - Megjelentek újabb adatmodellek (pl. objektumorientált), de tömegesen nem terjedtek el
 - Elterjedtek viszont az adattárházak
 - Tömegessé váltak az internetes, adatbázisokra épülő alkalmazások
 - A kezelendő adatmennyiség sokszorosára nőtt
- ➔ A relációs adatbázisok egyre több kihívással néztek szembe

2010 után

Nem relációs (NoSQL) adatmodellek megjelenése

- A legtöbb esetben egy régi ötlet újragondolását jelentik
- Nincs szabványos lekérdező nyelvük
- Részletesebben ld. 9-12. előadásokon

Napjainkban + Közeljövőben

Relációs és nem relációs adatbázisok együttélése

- A közöttük lévő határok elmosódnak (NewSQL)
 - Pl: CockroachDB, TiDB, Google Spanner
- Hibrid rendszerek
- Többmodelles adatbázis rendszerek
 - Pl: ArangoDB, CosmosDB, OrientDB

Napjainkban + Közeljövőben (folyt.)

- Felhős adatbázis rendszerek (DBaaS)
 - Adatbázisok: Google BigQuery, Azure SQL Database, AWS RDS
 - Serverless megoldások: Firestore (Google), Azure Cosmos DB
- Adattavak (Data Lake)
- Data Lakehouse
 - Ötvözik az adattavak és az adattárházak előnyeit, pl. Databricks, Snowflakes
- Adatstream feldolgozás
 - Pl. Kafka, Flink

- **AI által támogatott adatbázisok:** pl. Oracle Autonomous Database, amely mesterséges intelligenciával optimalizálja az adatbázis teljesítményét.
- **Generatív AI és SQL:** új AI eszközök (pl. ChatGPT) egyre jobban képesek SQL optimalizálásra és lekérdezések generálására.



ÓBUDAI EGYETEM
ÓBUDA UNIVERSITY

Gyakorlat



ÓBUDAI EGYETEM
ÓBUDA UNIVERSITY

A csoportosítás speciális lehetőségei

Részösszegek – GROUP BY ROLLUP

Az oszlopneveket és a NULL értéket kombinálva csoportosít, és megjeleníti a részösszegeket valamint végösszeget. Csoportosításkor a nem NULL oszlopok száma jobbról balra csökken

```
SELECT oszlopkifejezések*,  
        aggregálás  
FROM ...  
GROUP BY  
ROLLUP(oszlopkifejezések*)
```

* Többnyire oszlopnevek listáját jelenti

Pl:

```
SELECT oszlop1, oszlop2, oszlop3,  
        SUM(oszlop4)
```

FROM ...

GROUP BY ROLLUP (oszlop1, oszlop2, oszlop3)
esetén a következő csoportok jönnek létre:

- Oszlop1, oszlop2, oszlop3
- Oszlop1, oszlop2, NULL
- Oszlop1, NULL, NULL
- NULL, NULL, NULL

Részösszegek – GROUP BY ROLLUP - Példa

RAKTAR_KOD	KAT_ID	MEGYS	Készlet értéke
5	5	db	294000
5	5	NULL	294000
5	NULL	NULL	294000
6	9	db	1262600
6	9	NULL	1262600
6	NULL	NULL	1262600
9	5	db	73000
9	5	NULL	73000
9	9	db	11800
9	9	NULL	11800
9	NULL	NULL	84800
NULL	NULL	NULL	1641400

Példa

Készítsünk listát a raktáron lévő termékek összértékéről raktárkód, azon belül kategóriakód, majd mennyiségi egység szerinti bontásban! A lista jelenítse meg a részösszegeket és a végösszeget is! A listát szűrjük az 5-ös és 9-es azonosítójú kategóriára! A csoportosításnál a ROLLUP záradékot használjuk!

```

SELECT RAKTAR_KOD, KAT_ID, MEGYS,
       SUM(LISTAAR*KESZLET)
       AS 'Készlet értéke' FROM

```

Termek

WHERE kat_ID IN (5,9)

GROUP BY ROLLUP (RAKTAR_KOD, KAT_ID, MEGYS)

Részösszegek – GROUP BY CUBE

Az oszlopneveket és a NULL értéket kombinálva csoportosít, és megjeleníti a részösszegeket valamint végösszeget. A csoportosításhoz minden lehetséges kombinációt felhasznál.

```
SELECT oszlopkifejezések,  
        aggregálás  
FROM ...  
GROUP BY  
CUBE(oszlopkifejezések)
```

Pl:

```
SELECT oszlop1, oszlop2, oszlop3, SUM(oszlop4)  
FROM ...
```

GROUP BY CUBE (oszlop1, oszlop2, oszlop3) esetén a következő csoportok jönnek létre:

- Oszlop1, oszlop2, oszlop3
- Oszlop1, oszlop2, NULL
- Oszlop1, NULL, oszlop3
- Oszlop1, NULL, NULL
- NULL, oszlop2, oszlop3
- NULL, oszlop2, NULL
- NULL, NULL, oszlop3
- NULL, NULL, NULL

Részösszegek – GROUP BY CUBE- PÉLDA

RAKTAR_KOD	KAT_ID	MEGYS	Készlet értéke
5	5	db	294000
9	5	db	73000
NULL	5	db	367000
6	9	db	1262600
9	9	db	11800
NULL	9	db	1274400
NULL	NULL	db	1641400
NULL	NULL	NULL	1641400
5	NULL	db	294000
5	NULL	NULL	294000
6	NULL	db	1262600
6	NULL	NULL	1262600
9	NULL	db	84800
9	NULL	NULL	84800
5	5	NULL	294000
9	5	NULL	73000
NULL	5	NULL	367000
6	9	NULL	1262600
9	9	NULL	11800
NULL	9	NULL	1274400

Példa

Készítsünk listát a raktáron lévő termékek összértékéről raktárkód, azon belül kategóriakód, majd mennyiségi egység szerinti bontásban! A lista jelenítse meg a részösszegeket és a végösszeget is! A listát szűrjük az 5-ös és 9-es azonosítójú kategóriára! A csoportosításnál a CUBE záradékot használjuk!

```

SELECT RAKTAR_KOD, KAT_ID, MEGYS,
       SUM(LISTAAR*KESZLET)
       AS 'Készlet értéke'
FROM Termek
WHERE kat_ID IN (5,9)
GROUP BY CUBE (RAKTAR_KOD, KAT_ID, MEGYS)
  
```

A GROUP BY parancs kiegészítve a GROUPING SETS taggal lehetővé teszi, hogy többféle csoportosítást is megadjunk.

A csoportosításokat leíró
oszlopkifejezéseket egymás után,
zárójelek között , vesszővel
elválasztva kell megadni*

Pl:

```
SELECT oszlop1, oszlop2, SUM(oszlop3)
FROM table
GROUP BY
GROUPING SETS(( oszlop1, oszlop2), (oszlop1)
)
```

* A GROUPING SETS-ek az egyes csoportokra kiadott SELECT-ek UNION ALL-jainak alternatívái

GROUPING SETS - PÉLDA

szulev	nem	Átlagos életkor
NULL	F	35
NULL	N	49
1967	NULL	53
1975	NULL	45
1976	NULL	44
1980	NULL	40
1985	NULL	35
1997	NULL	23

Példa

Készítsünk listát az egyes ügyfelek átlagos életkoráról az ügyfél neme, illetve az ügyfél születési éve szerint csoportosítva! A listát szűrjük azon ügyfelekre, akik neve D-vel vagy E-vel kezdődik!
(Az életkor legyen a születési évtől a jelenlegi évig eltelt évek száma)

```

SELECT szulev, nem,
       AVG(YEAR(GETDATE())-SZULEV)
       AS 'Átlagos életkor'
FROM Ugyfel
WHERE NEV LIKE 'E%' OR NEV LIKE 'D%' GROUP BY
GROUPING SETS((SZULEV),(NEM))

```

A GROUPING fv. értéke 1, ha az adott oszlopkifejezés szerint aggregálás (sum, min, max stb.) van, különben 0

A GROUPING_ID fv. értéke a paraméterként megkapott oszlopkifejezések aggregációs szintjének száma*

- Használhatók a SELECT, a HAVING és az ORDER BY részekben, amennyiben a GROUP BY is specifikálva van
- Mindkét fv. legfontosabb alkalmazása, hogy megkülönböztessük a ROLLUP, CUBE és GROUPING SETS-ek által visszaadott NULL értékeket (részösszegek, végösszegek) a normál NULL értékektől
- Ha egy oszlop(kifejezés) szerint csoportosítunk, akkor a két fv. ugyanazt az értéket adja vissza

* A szintszám számítása úgy történik, hogy az oszloplistához egy bináris kódot rendelünk. Ennek i-ik eleme 1, ha az i-ik oszlop szerint van aggregálás, különben 0. A szintszám a bináris szám értéke lesz tízes számrendszerben. Pl: ha a csoport (oszlop1, oszlop2), és mindkettő szerint van aggregálás, akkor a szintszám binárisan 11, decimálisan 3.

GROUPING & Részösszeg, végösszeg

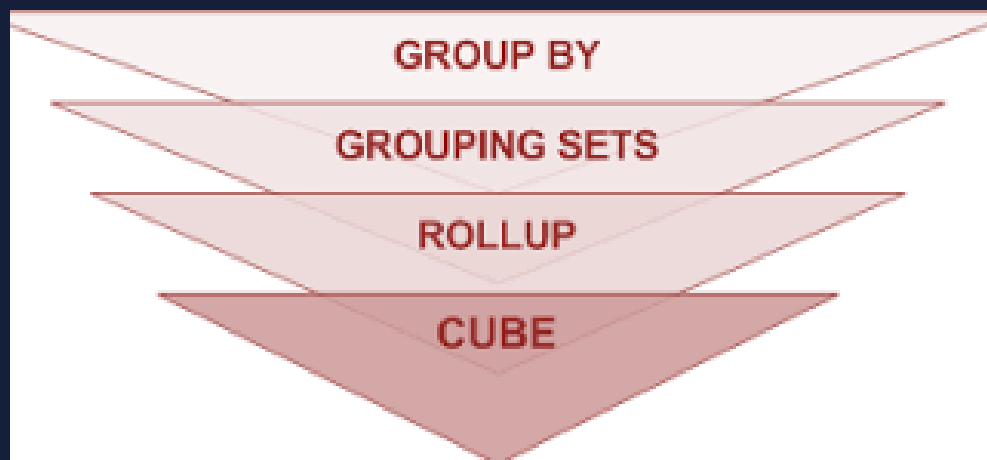
Pl: listázzuk, hogy melyik évben hány db terméket rendeltek meg! A lista megfelelően jelölve jelenítse meg a rendelések teljes összegét is!

```
SELECT
(
CASE GROUPING(YEAR(REND_DATUM))
WHEN 0 THEN CAST(YEAR(REND_DATUM)
AS nvarchar(4))
WHEN 1 THEN 'Összesen' END
)
AS ÉV,
COUNT(*) AS 'DB'
FROM Rendeles
GROUP BY ROLLUP(YEAR(REND_DATUM))
```

Pl: listázzuk, hogy naponta, azon belül fizetési mód szerint hány rendelés történt! A lista megfelelően jelölve jelenítse meg a részösszegeket és a végösszeget is!

```
SELECT IIF(GROUPING(REND_DATUM)=1, 'Összesen',
CAST(REND_DATUM AS nvarchar(10)))
AS 'Rendelés dátuma',
(
CASE GROUPING_ID(REND_DATUM, FIZ_MOD)
WHEN 0 THEN FIZ_MOD
WHEN 1 THEN '**Fizetési módok összesen**'
WHEN 3 THEN 'Összesen'
END) as 'FIZ_MOD',
COUNT(*) as 'DB'
FROM Rendeles
GROUP BY ROLLUP(REND_DATUM, FIZ_MOD)
```

A GROUPING SETS, GROUPING /
GROUPING_ID fv.
és a ROLLUP / CUBE együtt is
használhatók



```
SELECT CASE
    WHEN GROUPING(szulev)=0
    THEN CAST(SZULEV AS nvarchar(4))
    ELSE 'Minden év'
END AS 'Születési év', CASE
    WHEN GROUPING(nem)=1
    THEN 'Minden nem'
    ELSE NEM
END AS 'Nem',
    AVG(YEAR(GETDATE())-SZULEV)
    AS 'Átlagos életkor'
FROM Ugyfel
WHERE NEV LIKE 'E%' OR NEV LIKE 'D%' GROUP BY
GROUPING SETS(ROLLUP(SZULEV),
CUBE(nem))
```

WebShop adatbázis szerkezete

TERMEK_KATEGORIA		
KAT_ID	NUMERIC(5)	NOT_NULL
KAT_NEV	VARCHAR(32)	NOT_NULL
SZULO_KAT	NUMERIC(5)	

RAKTAR		
RAKTAR_KOD	NUMERIC(3)	NOT_NULL
RAKTAR_NEV	VARCHAR(48)	NOT_NULL
RAKTAR_CIM	VARCHAR(128)	NOT_NULL

TERMEK		
TERMEKKOD	VARCHAR(16)	NOT_NULL
MEGNEVEZES	VARCHAR(64)	NOT_NULL
KAT_ID	NUMERIC(5)	
LISTAAR	NUMERIC(8,2)	
LEIRAS	TEXT	
RAKTAR_KOD	NUMERIC(3)	NOT_NULL
KESZLET	NUMERIC(7,2)	
MEGYS	VARCHAR(16)	
FELVITTE	VARCHAR(16)	NOT_NULL
FELVITEL	DATE	NOT_NULL

UGYFEL		
LOGIN	VARCHAR(32)	NOT_NULL
EMAIL	VARCHAR(64)	NOT_NULL
NEV	VARCHAR(64)	NOT_NULL
SZULEV	NUMERIC(4)	
NEM	VARCHAR(1)	
CIM	VARCHAR(128)	

RENDELESI_TETEL		
SORSZAM	NOT_NULL	NUMERIC(7)
TERMEKKOD	NOT_NULL	VARCHAR(16)
EGYSEGAR	NUMERIC(8,2)	
MENNYISEG	NOT_NULL	NUMERIC(7,2)

RENDELES		
SORSZAM	NUMERIC(7)	NOT_NULL
LOGIN	VARCHAR(32)	NOT_NULL
REND_DATUM	DATE	NOT_NULL
SZALL_DATUM	DATE	
SZALL_CIM	VARCHAR(128)	
SZALL_MOD	VARCHAR(24)	
FIZ_MOD	VARCHAR(24)	
SZAMLA_CIM	VARCHAR(128)	